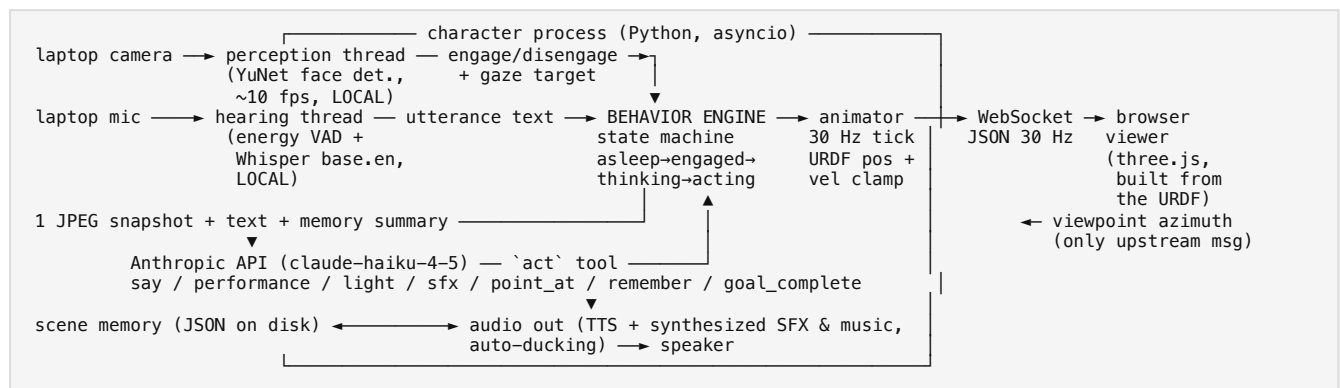


Technical Note — Ikea Lamp

I built a live desk-lamp character around the supplied 5-DOF URDF. It sees through the laptop camera, listens through the mic, talks through the speaker, and performs with motion, light, chirps, and music in a 3D body I render in the browser. The goal was one coherent character, not a pile of demos, so one behavior engine owns everything and every action Ikea Lamp takes is a single combined gesture: speech + motion + light + sound together.

1. Architecture and data flow



One process owns all state. The browser is just a monitor pointed at the robot's body. The URDF is the single source of truth: I parse it once, ship the description to the viewer (which builds the 3D scene from it), and the animator enforces the same file's joint position **and velocity** limits on every tick. Nothing about the robot is duplicated by hand.

2. Protocol

JSON over one WebSocket. Downstream (authoritative): robot once on connect, state at 30 Hz (joint positions, hop offset, light color/intensity), hud (behavior state + captions), camera at 5 Hz (annotated webcam thumbnail so "what the lamp sees" is visible). Upstream, exactly one message: viewpoint — the orbit camera's azimuth, which the character treats as "where my audience is" and turns toward through real joints. The viewer is a swappable display; the same protocol could feed a physics sim or firmware.

3. Model-to-action (the VLA boundary)

Per utterance, Claude gets three things: the transcribed text, **one** downscaled JPEG snapshot, and a compact summary of scene memory. It must answer through a forced, strictly-validated `act` tool — the character's whole action vocabulary: `say`, a named motion performance, light color/intensity, an SFX, an optional `point_at(x, y)` in **image-normalized coordinates**, `remember[]` (objects worth storing: name, description, position), and `goal_complete`. So the model decides *what* (semantic actions grounded in pixels) and the controller decides *how* (maps image coordinates to joint angles, clamps to URDF limits). The model never outputs joint values. Goal-directed actions run as a loop: `act` → `move` → `take a fresh snapshot` → Claude checks its own aim → only then `goal_complete`. Memory is a JSON store Claude writes to via `remember` and reads back through the summary, so it can answer "where was my mug?" after the mug leaves the frame.

4. Simulation

Kinematic, no physics engine. The real robot is a desk-mounted servo arm — its dynamics are the servos, so a physics sim would add cost and failure modes without adding believability. Physical honesty lives where it matters: every tick clamps to URDF position and velocity limits (85% derating), motion is a performance layer (authored keyframes with spring easing) + a breathing layer (never fully still) + a gaze layer, all summed then clamped. I chose a custom three.js viewer over PyBullet/MuJoCo for expressive rendering (a real spotlight, an emissive Edison bulb, a themed body) and because the WebSocket boundary makes the architecture explicit. One deliberate liberty: the lamp hops during "excited". That is a display-only channel with a ballistic profile, clearly separated from the joints; a real desk-mounted unit would drop it. The URDF itself is unmodified — the viewer restyles materials and hides the camera-marker nub, that's all.

5. Deployment (Ubuntu 24.04, 4 cores, 8 GB, no GPU)

Plain Python venv + a handful of `apt` packages (README has exact commands). No containers needed at this scale. Everything CPU-heavy was sized for the target: YuNet face detection (230 KB ONNX) at ~10 fps, faster-whisper `base.en` int8 (~0.5 s per utterance on CPU), 30 Hz animation. TTS auto-selects: `piper` (local neural voice) on Linux, `say` on my Mac dev machine. The viewer vendors all JS, so the only network dependency is the Anthropic

API. Camera/mic/speaker are selectable from the viewer (thumbnails for cameras — OpenCV gives no names) and persisted, which mattered on macOS where the iPhone kept hijacking camera index 0.

Cloud data policy: face detection, VAD, and speech-to-text run locally. What leaves the machine: utterance text, one JPEG per exchange, the memory summary. No audio or video streams, ever. Claude Haiku over a bigger model because ~2–3 s replies matter more to a live character than depth — model is a config knob.

6. Measurements (MacBook dev machine, from built-in instrumentation → `metrics.json`)

Metric	Measured
Engagement: face appears → wake begins	~0.8 s (0.7 s debounce + ~10 fps detection); disengage after 2.5 s absence. Zero false wakes over multi-minute idle runs; no flicker thanks to debounce both ways
STT latency (utterance end → text)	0.45 s (base.en int8, CPU)
LLM latency (snapshot + text → action)	2.6–2.9 s (claude-haiku-4-5, forced tool call)
Utterance end → reply <i>starts</i>	≈ 3.5 s (STT + LLM + gesture kickoff)
Utterance end → reply <i>finished</i>	7.7 s measured — the remainder is TTS speaking a 1–2 sentence reply (2.8–4.8 s)
CPU, whole process, engaged + tracking	~60% of one core (peak 74%)
Memory (RSS)	~700 MB (peak 860) — Whisper + OpenCV + numpy audio buffers; fits the 8 GB target comfortably
Camera pipeline	10.5 fps sustained; animation 30 Hz

The 2.6 s LLM hop is the dominant latency; streaming a text-first response would cut perceived latency but breaks the "one atomic action" contract, so I kept it and hid the wait with a "thinking" pose + pulsing light instead.

7. Timebox

About 7 hours across two days, AI-assisted throughout — the architecture, behavior design, measurements, and decisions here are mine.

8. Known limitations / intentionally left out

- Aiming uses a fixed camera→joint mapping (laptop camera ≈ lamp's forward view), not calibrated hand-eye geometry. Re-observation covers coarse aim.
- Engagement is face-presence: YuNet's frontal bias approximates "looking toward" but there's no true gaze estimation. Single person; no face ID.
- Energy VAD works in normal rooms (asymmetric floor tracking absorbs fans and the music bed); very noisy spaces would want a neural VAD (silero) — cut for scope.
- No physics/collision, no IK: expressive poses are authored and pointing is a 2-DOF mapping. Fine for a lamp; wouldn't scale to a manipulator.
- Ubuntu setup is written and dependency-checked but not run on a clean 24.04 machine — I developed and measured on macOS.